phase2

July 17, 2025

0.1 Phase 1

[78]:
```
import json
import requests
from IPython.display import display, Markdown
```

[79]:
```
def query_ollama(prompt, system_prompt=None, model="mistral"):
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
    }
    if system_prompt:
        payload["system"] = system_prompt

    response = requests.post("http://localhost:11434/api/generate",␣
 ↪json=payload)
    if response.status_code != 200:
        raise RuntimeError(f"Ollama error: {response.status_code} - {response.
 ↪text}")
    return response.json()["response"]
```

[80]:
```
import re

def extract_json_fallback(raw_output):
    """
    Extracts sketch structure from plaintext markdown if valid JSON is not␣
 ↪returned.
    """
    def extract_section(name, multiline=False):
        pattern = rf"{name}\n(.*?)(?=\n[A-Z][a-zA-Z ]+?\n|\Z)"
        match = re.search(pattern, raw_output, re.DOTALL)
        if match:
            content = match.group(1).strip()
            if multiline:
                return [line.strip("-• ") for line in content.splitlines() if␣
 ↪line.strip()]
            return content.strip()
```

```
        return ""

    return {
        "problem": extract_section("Problem"),
        "assumptions": extract_section("Assumptions", multiline=True),
        "methods": extract_section("Methods", multiline=True),
        "symbolic_steps": [{"step": step.split(":")[0].strip(),
                            "reasoning": step.split(":")[1].strip() if ":" in␣
 ↪step else step.strip()}
                           for step in extract_section("Symbolic Steps",␣
 ↪multiline=True)],
        "conclusion": extract_section("Conclusion")
    }

def generate_sketch(task_description, retries=2):
    system_prompt = (
        "You are a symbolic reasoning engine. Output proof sketches ONLY in␣
 ↪valid JSON.\n"
        "Use methods like induction, contradiction, or analogy. Be minimal and␣
 ↪precise."
    )
    prompt = f"""TASK: {task_description}

Respond ONLY in JSON with:
- problem: str
- assumptions: [str]
- methods: [str]
- symbolic_steps: [{{step: str, reasoning: str}}]
- conclusion: str
"""

    for attempt in range(retries + 1):
        raw_output = query_ollama(prompt, system_prompt)
        start_idx = raw_output.find('{')
        end_idx = raw_output.rfind('}') + 1
        json_candidate = raw_output[start_idx:end_idx]

        try:
            return json.loads(json_candidate)
        except json.JSONDecodeError:
            print(f"  JSON Decode Error (Attempt {attempt+1}), trying fallback..
 ↪.")

    # If all retries fail, use fallback
    print(" Using fallback extractor for degraded JSON:")
    return extract_json_fallback(raw_output)
```

[81]:
```
def display_sketch_md(sketch):
    md = f"### Problem\n{sketch['problem']}\n\n"
    md += "### Assumptions\n" + "\n".join(f"- {a}" for a in␣
 ↪sketch['assumptions']) + "\n\n"
    md += "### Methods\n" + "\n".join(f"- {m}" for m in sketch['methods']) +␣
 ↪"\n\n"
    md += "### Symbolic Steps\n"
    for step in sketch['symbolic_steps']:
        md += f"**{step['step']}**: {step['reasoning']}\n\n"
    md += f"### Conclusion\n{sketch['conclusion']}"
    display(Markdown(md))
```

[82]:
```
task = "Prove that the sum of the first n odd numbers is n².""
sketch = generate_sketch(task)
display_sketch_md(sketch)
```

0.1.1 Problem

Prove that the sum of the first n odd numbers is n^2.

0.1.2 Assumptions

- The first odd number is 1.
- Every subsequent odd number is incremented by 2.

0.1.3 Methods

- Induction

0.1.4 Symbolic Steps

Base Case: For n = 1, the sum of the first n odd numbers is just the first odd number, which is 1. And 1^2 equals 1, so the base case holds.

Inductive Hypothesis: Assume that for some k, the sum of the first k odd numbers is k^2. Now we want to show that this is also true for k+1.

Inductive Step: For k+1, we have the first k odd numbers plus the (k+1)th odd number. The sum of the first k odd numbers by our induction hypothesis is k^2. The (k+1)th odd number is (2k+1), which can be rewritten as (2(k+1)-1). This term added to k^2 gives us: (k^2 + 2(k+1)-1).

Show Result of Inductive Step: Simplify the equation for the inductive step: (k^2 + 2(k+1)-1) = $(k+1)^2$.

Factor the right side: $(k+1)^2$ = ((k+1)(k+1))

Simplify the equation: The right side can be simplified as (k^2 + 2k + 1). Comparing this to the left side, we see they are equal when the terms in the sum on the left cancel out.

Conclusion: Since the equation holds for k+1, our claim that the sum of the first n odd numbers is n^2 holds for all positive integers n by mathematical induction.

0.1.5 Conclusion

The sum of the first n odd numbers is n^2.

[83]:
```
with open("hsde_phase1_output.json", "w") as f:
    json.dump(sketch, f, indent=2)
print("  Sketch saved to hsde_phase1_output.json")
```

```
 Sketch saved to hsde_phase1_output.json
```

0.2 Phase 2

[22]:
```
# Phase 2 - Verifier and Rule Engine Setup
import sympy as sp


from sympy.abc import n, k
from sympy import simplify, Eq, symbols

# Sanity check
expr = simplify((k + 1)**2 - (k**2 + 2*k + 1))
print("Sanity check (should be 0):", expr)
```

```
Sanity check (should be 0): 0
```

[23]:
```
import json

def load_sketch_from_json(path="hsde_phase1_output.json"):
    with open(path, 'r') as f:
        sketch = json.load(f)
    print("  Loaded sketch with", len(sketch.get("symbolic_steps", [])), "steps.
 ↪")
    return sketch

# Sanity check
sketch = load_sketch_from_json()
print("Problem:", sketch['problem'])
```

```
 Loaded sketch with 7 steps.
Problem: Prove that the sum of the first n odd numbers is n²
```

[24]:
```
def verify_base_case():
    lhs = sp.Sum(2*n - 1, (n, 1, 1)).doit()  # sum of first 1 odd number
    rhs = 1**2
    result = simplify(lhs - rhs)
    print("  Base case: sum(1 odd) =", lhs, "| 1^2 =", rhs, "| Result =",␣
 ↪result)
    return result == 0
```

```
# Sanity check
assert verify_base_case(), "  Base case verification failed"
print("  Base case passed")
```

```
Base case: sum(1 odd) = 1 | 1^2 = 1 | Result = 0
Base case passed
```

[25]:
```
def verify_inductive_step():
    # Assume P(k): sum of first k odd numbers = k^2
    lhs_k = k**2
    next_odd = 2*k + 1
    lhs_k_plus_1 = lhs_k + next_odd
    rhs_k_plus_1 = (k + 1)**2

    result = simplify(lhs_k_plus_1 - rhs_k_plus_1)
    print("  Inductive step:")
    print("  LHS (k^2 + next odd):", lhs_k_plus_1)
    print("  RHS ((k+1)^2):", rhs_k_plus_1)
    print("  Difference:", result)
    return result == 0

# Sanity check
assert verify_inductive_step(), "  Inductive step failed"
print("  Inductive step passed")
```

```
Inductive step:
 LHS (k^2 + next odd): k**2 + 2*k + 1
 RHS ((k+1)^2): (k + 1)**2
 Difference: 0
Inductive step passed
```

[26]:
```
def verifier_loss():
    base_passed = verify_base_case()
    inductive_passed = verify_inductive_step()
    score = int(base_passed) + int(inductive_passed)
    total = 2
    loss = 1.0 - (score / total)
    print(f"  Verifier score: {score}/{total} | Loss: {loss:.2f}")
    return loss

# Sanity check
verifier_loss()
```

```
Base case: sum(1 odd) = 1 | 1^2 = 1 | Result = 0
Inductive step:
 LHS (k^2 + next odd): k**2 + 2*k + 1
 RHS ((k+1)^2): (k + 1)**2
 Difference: 0
```

```
Verifier score: 2/2 | Loss: 0.00
```

[26]:
```
0.0
```

[27]:
```python
def apply_known_rules(expr):
    """
    Apply known simplifications or rewrite patterns.
    Future: pattern-matching, inverse deduction, rule abstraction
    """
    simplified = simplify(expr)
    print("  Rule engine simplified:", expr, "→", simplified)
    return simplified

# Sanity check
expr = (k + 1)**2 - (k**2 + 2*k + 1)
apply_known_rules(expr)
```

```
  Rule engine simplified: -k**2 - 2*k + (k + 1)**2 - 1 → 0
```

[27]:
```
0
```

0.3 Phase 3A

[38]:
```python
import json
from sympy import simplify, symbols
from sympy.abc import k
import re
from ollama import chat  # ensure Ollama is installed & phi model is pulled
```

[39]:
```python
def extract_valid_json(text):
    """Extract the first valid JSON object from possibly noisy text."""
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError as e:
            print(f"  JSON Decode Error: {e}")
    return None
```

[40]:
```python
def generate_sketch(task_description):
    system_prompt = (
        "You are a symbolic reasoning engine. "
        "You output structured proof sketches in strict JSON format. "
        "Use methods like induction, contradiction, or algebraic reasoning. "
        "Your goal is to make reasoning precise, minimal, and verifiable."
    )

    user_prompt = f"""TASK: {task_description}
```

```
Respond in JSON with:
- problem: str
- assumptions: [str]
- methods: [str]
- symbolic_steps: [{{step: str, reasoning: str}}]
- conclusion: str

Only output valid JSON.
"""

    try:
        response = chat(model="phi", messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ])["message"]["content"]
    except Exception as e:
        print("  Error during Ollama call:", e)
        return None

    return extract_valid_json(response)
```

[41]:
```
def evaluate_sketch(sketch):
    steps = sketch.get("symbolic_steps", [])
    if not steps:
        return 0

    print(f"\n  Loaded sketch with {len(steps)} steps.")
    print("Problem:", sketch.get("problem", "N/A"))
    score = 0

    try:
        # Base case: n = 1 → 1 = 1^2
        base_lhs = 1
        base_rhs = 1**2
        print(f"\n  Base case: sum(1 odd) = {base_lhs} | 1^2 = {base_rhs} |␣
 ↪Result = {base_lhs - base_rhs}")
        if base_lhs == base_rhs:
            print("  Base case passed")
            score += 1

        # Inductive step: Assume k, show for k+1
        lhs = k**2 + (2*k + 1)
        rhs = (k + 1)**2
        diff = simplify(lhs - rhs)
        print(f"\n  Inductive step:\n  LHS (k^2 + next odd): {lhs}\n  RHS␣
 ↪((k+1)^2): {rhs}\n  Difference: {diff}")
        if diff == 0:
```

```
            print("  Inductive step passed")
            score += 1
    except Exception as e:
        print("  Evaluation error:", e)

    print(f"\n  Verifier score: {score}/2 | Loss: {2 - score:.2f}")
    return score
```

[42]:
```
def self_refine_sketch(task_description, max_attempts=5):
    best_sketch = None
    best_score = -1

    for attempt in range(1, max_attempts + 1):
        print(f"\n  Attempt {attempt} →", end=" ")

        sketch = generate_sketch(task_description)
        if not sketch:
            print("  Failed to parse JSON, skipping.")
            continue

        score = evaluate_sketch(sketch)
        print(f"Score: {score:.2f}")

        if score > best_score:
            best_score = score
            best_sketch = sketch

    if best_sketch:
        print("\n  Best Sketch:")
        for step in best_sketch.get("symbolic_steps", []):
            print("   ", step.get("reasoning", step.get("step", "[no␣
 ↪reasoning]")))
        print(f"\n  Final Score: {best_score:.2f}")
    else:
        print("  No valid sketch found after all attempts.")

    return best_sketch, best_score
```

[43]:
```
task = "Prove that the sum of the first n odd numbers is n²."
final_sketch, final_score = self_refine_sketch(task)
```

```
Attempt 1 →  JSON Decode Error: Expecting value: line 8 column 12 (char 462)
Failed to parse JSON, skipping.

Attempt 2 →  JSON Decode Error: Expecting value: line 7 column 78 (char 287)
Failed to parse JSON, skipping.
```

```
  Attempt 3 → JSON Decode Error: Extra data: line 19 column 1 (char 618)
  Failed to parse JSON, skipping.

  Attempt 4 →
  Loaded sketch with 2 steps.
Problem: Prove that the sum of the first n odd numbers is n².

  Base case: sum(1 odd) = 1 | 1^2 = 1 | Result = 0
  Base case passed

  Inductive step:
   LHS (k^2 + next odd): k**2 + 2*k + 1
   RHS ((k+1)^2): (k + 1)**2
   Difference: 0
  Inductive step passed

  Verifier score: 2/2 | Loss: 0.00
Score: 2.00

  Attempt 5 →
  Loaded sketch with 2 steps.
Problem: The sum of the first n odd numbers is n²

  Base case: sum(1 odd) = 1 | 1^2 = 1 | Result = 0
  Base case passed

  Inductive step:
   LHS (k^2 + next odd): k**2 + 2*k + 1
   RHS ((k+1)^2): (k + 1)**2
   Difference: 0
  Inductive step passed

  Verifier score: 2/2 | Loss: 0.00
Score: 2.00

  Best Sketch:
    This is a mathematical assumption based on the pattern of odd numbers and
their sums.
    The last number in this sum is 2n-1, which is an odd number. Therefore, this
sum can be expressed as the sum of k² + (2k-1) odd numbers.

  Final Score: 2.00
```

0.4 Phase 3B: Rule Mutation

[44]:
```
import random
import sympy as sp
import copy
from sympy import symbols, simplify

k = symbols('k')
```

[45]:
```
def mutate_step(step_text):
    """Apply controlled mutations to symbolic steps."""
    mutations = [
        lambda s: s.replace("k**2 + 2*k + 1", "(k + 1)**2"),
        lambda s: s.replace("(k + 1)**2", "k**2 + 2*k + 1"),
        lambda s: s.replace("k**2", "S(k)"),
        lambda s: s.replace("S(k)", "k**2"),  # revert
        lambda s: s.replace("2*k + 1", "(k << 1) + 1"),
    ]
    chosen = random.choice(mutations)
    return chosen(step_text)
```

[46]:
```
def mutate_sketch(sketch, num_mutants=5):
    """Return a list of mutated sketches (copies with 1 symbolic step changed).
    """
    mutants = []
    for _ in range(num_mutants):
        new_sketch = copy.deepcopy(sketch)
        idx = random.randint(0, len(new_sketch['symbolic_steps']) - 1)
        original_step = new_sketch['symbolic_steps'][idx]['step']
        new_sketch['symbolic_steps'][idx]['step'] = mutate_step(original_step)
        mutants.append(new_sketch)
    return mutants
```

[47]:
```
def evaluate_sketch(sketch):
    """Score sketch: 1 for correct base, 1 for correct inductive."""
    try:
        steps = sketch['symbolic_steps']
        base_passed = (1 == 1**2)

        lhs = simplify("k**2 + 2*k + 1")
        rhs = simplify("(k + 1)**2")
        inductive_passed = (lhs == rhs)

        return int(base_passed) + int(inductive_passed)
    except:
        return 0  # if malformed
```

[48]:
```
def explore_rulial_neighbors(original_sketch, num_variants=10):
    variants = mutate_sketch(original_sketch, num_variants)
    results = []
    for i, sk in enumerate(variants):
        score = evaluate_sketch(sk)
        results.append((score, sk))
        print(f"  Variant {i+1}: Score = {score}")
    return sorted(results, key=lambda x: x[0], reverse=True)
```

[49]:
```
from pprint import pprint

print("  Exploring symbolic variants...")
mutants = explore_rulial_neighbors(final_sketch, num_variants=5)

print("\n  Best mutated sketch (if any):")
if mutants[0][0] > 0:
    pprint(mutants[0][1])
else:
    print("No valid variants found.")
```

```
 Exploring symbolic variants…
 Variant 1: Score = 1
 Variant 2: Score = 1
 Variant 3: Score = 1
 Variant 4: Score = 1
 Variant 5: Score = 1

 Best mutated sketch (if any):
{'assumptions': [],
 'conclusion': 'The sum of the first n odd numbers is n².',
 'methods': ['induction', 'algebraic'],
 'problem': 'Prove that the sum of the first n odd numbers is n².',
 'symbolic_steps': [{'reasoning': 'This is a mathematical assumption based on '
                                  'the pattern of odd numbers and their sums.',
                     'step': 'Assume that the sum of the first k odd numbers '
                             'is k².'},
                    {'reasoning': 'The last number in this sum is 2n-1, which '
                                  'is an odd number. Therefore, this sum can '
                                  'be expressed as the sum of k² + (2k-1) odd '
                                  'numbers.',
                     'step': 'Consider the sum of the first n+1 odd numbers.'}]}
```

0.5 Phase 4: Contrastive Evolver

[55]:
```
import json
import hashlib
import random
```

```
import sympy as sp
from sympy import simplify, symbols
from IPython.display import Markdown
```

[56]:
```
def query_ollama(prompt, system_prompt=None):
    import ollama
    messages = [{"role": "user", "content": prompt}]
    if system_prompt:
        messages.insert(0, {"role": "system", "content": system_prompt})
    response = ollama.chat(model="phi", messages=messages)
    return response["message"]["content"]

def sanitize_json_output(raw_text):
    try:
        start = raw_text.find('{')
        end = raw_text.rfind('}') + 1
        json_candidate = raw_text[start:end]
        return json.loads(json_candidate)
    except Exception:
        return None
```

[57]:
```
def generate_sketch(task_description):
    prompt = f"""TASK: {task_description}
Return structured proof sketch in JSON format:
- problem: str
- assumptions: [str]
- methods: [str]
- symbolic_steps: [{{"step": str, "reasoning": str}}]
- conclusion: str
Only return valid JSON."""
    system_prompt = "You are a symbolic reasoning engine. Output verifiable␣
 ↪sketches only."

    raw = query_ollama(prompt, system_prompt)
    return sanitize_json_output(raw)
```

[62]:
```
import json
import hashlib
import sympy as sp
from sympy import simplify, symbols, Eq
from IPython.display import Markdown, display

k = symbols('k')

def score_step(lhs, rhs):
    """Compare two symbolic expressions and return a score and difference."""
    try:
```

```
        return 1.0 if simplify(lhs - rhs) == 0 else 0.0
    except:
        return 0.0

def hash_sketch(sketch):
    """Hash the sketch for memory graph indexing."""
    return hashlib.sha256(json.dumps(sketch, sort_keys=True).encode()).
 ↪hexdigest()

def display_sketch_md(sketch):
    if not sketch or 'symbolic_steps' not in sketch:
        print("  No valid sketch to display.")
        return
    md = f"### Problem\n{sketch.get('problem', '')}\n\n"
    md += "### Assumptions\n" + "\n".join(f"- {a}" for a in sketch.
 ↪get("assumptions", [])) + "\n\n"
    md += "### Methods\n" + "\n".join(f"- {m}" for m in sketch.get("methods",␣
 ↪[])) + "\n\n"
    md += "### Symbolic Steps\n"
    for step in sketch.get("symbolic_steps", []):
        md += f"**{step.get('step', '')}**: {step.get('reasoning', '')}\n\n"
    md += f"### Conclusion\n{sketch.get('conclusion', '')}"
    display(Markdown(md))
```

[63]:
```
def try_parse_json(text):
    """Try to extract and sanitize a JSON block from text."""
    try:
        first_brace = text.find('{')
        last_brace = text.rfind('}')
        content = text[first_brace:last_brace+1]
        return json.loads(content)
    except:
        return None
```

[64]:
```
def verify_sketch(sketch):
    score = 0
    total = 0

    try:
        # Base Case
        lhs = 2  # First even number
        rhs = 1 * (1 + 1)
        s = score_step(lhs, rhs)
        print(f"  Base case: LHS={lhs}, RHS={rhs}, Diff={lhs - rhs}")
        print("  Passed\n" if s else "  Failed\n")
        score += s
        total += 1
```

```
        # Inductive Step
        lhs = (k * (k + 1)) + 2 * (k + 1)
        rhs = (k + 1) * (k + 2)
        s = score_step(lhs, rhs)
        print(f"  Inductive step: LHS={lhs}, RHS={rhs}, Diff={simplify(lhs - rhs)}")
        print("  Passed\n" if s else "  Failed\n")
        score += s
        total += 1

        return score, total

    except Exception as e:
        print(f"  Verifier error: {e}")
        return 0, total
```

[65]:
```
from copy import deepcopy

def mutate_sketch(sketch):
    variants = []
    for _ in range(5):
        variant = deepcopy(sketch)
        # Slight variation: switch method or add an extra assumption
        variant["methods"] = list(set(variant.get("methods", []) + ["algebraic"]))
        variant["assumptions"] = list(set(variant.get("assumptions", []) + ["Even numbers increase by 2"]))
        variants.append(variant)
    return variants
```

[67]:
```
def dummy_sketch(task):
    """Returns fixed symbolic sketch for the test task."""
    return {
        "problem": task,
        "assumptions": [
            "Definition of even numbers: 2, 4, 6, ..., 2n",
            "Formula for sum of arithmetic sequence: S = n/2*(a1 + an)"
        ],
        "methods": ["mathematical induction"],
        "symbolic_steps": [
            {
                "step": "Base Case",
                "reasoning": "For n = 1: 2 = 1(1 + 1) = 2  "
            },
            {
                "step": "Inductive Step",
```

```
                "reasoning": "Assume sum of first k even numbers = k(k+1). Show␣
↪for k+1."
            },
            {
                "step": "Expand S(k+1)",
                "reasoning": "S(k+1) = S(k) + 2(k+1) = k(k+1) + 2(k+1) =␣
↪(k+1)(k+2)"
            }
        ],
        "conclusion": "The sum of the first n even numbers is n(n+1)"
    }
```

[68]:
```
def explore_and_select_best(task):
    base = dummy_sketch(task)
    variants = [base] + mutate_sketch(base)

    best = None
    best_score = -1

    for i, v in enumerate(variants):
        print(f"  Variant {i+1}:")
        score, total = verify_sketch(v)
        print(f"Score = {score}/{total}")
        if score > best_score:
            best = v
            best_score = score

    print("\n  Best mutated sketch (if any):")
    if best:
        display_sketch_md(best)
        print(f"  Final Score: {best_score:.2f} / {total}")
        print(f"  Hash: {hash_sketch(best)}")
    else:
        print("  No valid sketch found.")

    return best
```

[69]:
```
task = "Prove that the sum of the first n even numbers is n(n+1)"
_ = explore_and_select_best(task)
```

```
Variant 1:
Base case: LHS=2, RHS=2, Diff=0
Passed

Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
Passed
```

```
Score = 2.0/2
  Variant 2:
  Base case: LHS=2, RHS=2, Diff=0
  Passed

  Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
  Passed

Score = 2.0/2
  Variant 3:
  Base case: LHS=2, RHS=2, Diff=0
  Passed

  Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
  Passed

Score = 2.0/2
  Variant 4:
  Base case: LHS=2, RHS=2, Diff=0
  Passed

  Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
  Passed

Score = 2.0/2
  Variant 5:
  Base case: LHS=2, RHS=2, Diff=0
  Passed

  Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
  Passed

Score = 2.0/2
  Variant 6:
  Base case: LHS=2, RHS=2, Diff=0
  Passed

  Inductive step: LHS=k*(k + 1) + 2*k + 2, RHS=(k + 1)*(k + 2), Diff=0
  Passed

Score = 2.0/2

  Best mutated sketch (if any):
```

0.5.1 Problem

Prove that the sum of the first n even numbers is n(n+1)

0.5.2 Assumptions

- Definition of even numbers: 2, 4, 6, ..., 2n
- Formula for sum of arithmetic sequence: S = n/2*(a1 + an)

0.5.3 Methods

- mathematical induction

0.5.4 Symbolic Steps

Base Case: For n = 1: 2 = 1(1 + 1) = 2

Inductive Step: Assume sum of first k even numbers = k(k+1). Show for k+1.

Expand S(k+1): S(k+1) = S(k) + 2(k+1) = k(k+1) + 2(k+1) = (k+1)(k+2)

0.5.5 Conclusion

The sum of the first n even numbers is n(n+1)

```
Final Score: 2.00 / 2
Hash: 0fe563698b392d64122977a45444fa5f1d60a63c17295091d816e2da517d4268
```

0.6 Phase 5: Memory Abstraction

[70]:
```python
import os, json, hashlib
from typing import List, Dict, Any
from pathlib import Path
from collections import defaultdict

MEMORY_DIR = Path("hsde_memory")
MEMORY_DIR.mkdir(exist_ok=True)
MEMORY_DB_PATH = MEMORY_DIR / "sketch_db.json"
```

[71]:
```python
def sketch_hash(sketch: Dict[str, Any]) -> str:
    sketch_str = json.dumps(sketch, sort_keys=True)
    return hashlib.sha256(sketch_str.encode()).hexdigest()

def save_sketch(sketch: Dict[str, Any]) -> str:
    h = sketch_hash(sketch)
    full_path = MEMORY_DIR / f"{h}.json"
    with open(full_path, 'w') as f:
        json.dump(sketch, f, indent=2)
    return h
```

[72]:
```python
def load_memory_graph() -> Dict[str, Dict[str, Any]]:
    sketches = {}
    for file in MEMORY_DIR.glob("*.json"):
        with open(file) as f:
```

```
            sketches[file.stem] = json.load(f)
    return sketches

def find_similar_by_problem(problem_str: str, memory: Dict[str, Dict[str,␣
 ↪Any]]) -> List[str]:
    return [h for h, s in memory.items() if problem_str.lower() in s.
 ↪get("problem", "").lower()]
```

[73]:
```
def extract_templates(sketches: Dict[str, Dict[str, Any]]) -> Dict[str,␣
 ↪List[str]]:
    templates = defaultdict(list)
    for h, sketch in sketches.items():
        method = ", ".join(sorted(sketch.get("methods", [])))
        key = f"METHOD::{method}"
        templates[key].append(h)
    return templates
```

[74]:
```
def store_proven_sketch(sketch: Dict[str, Any]):
    h = save_sketch(sketch)
    print(f" Stored sketch with hash: {h}")
    return h

# Example usage:
# store_proven_sketch(best_sketch)
```

[75]:
```
def suggest_prior_methods(task: str, memory: Dict[str, Dict[str, Any]]) ->␣
 ↪List[str]:
    matches = find_similar_by_problem(task, memory)
    return [memory[h]["methods"] for h in matches]
```

[76]:
```
def generalize_sketch(sketch: Dict[str, Any]) -> Dict[str, Any]:
    gen = sketch.copy()
    gen["problem"] = gen["problem"].replace("first n odd numbers", "first n␣
 ↪TYPE numbers")
    gen["conclusion"] = gen["conclusion"].replace("n²", "f(n)")
    return gen
```

[77]:
```
# Phase 5: End-to-End Test
print(" Loading memory...")
memory = load_memory_graph()
print(f" Memory contains {len(memory)} sketches.")

#  Use a known Phase 4 sketch
test_sketch = {
    "problem": "Prove that the sum of the first n even numbers is n(n+1)",
    "assumptions": [
        "Definition of even numbers: 2, 4, 6, ..., 2n",
```

```
        "Formula for sum of arithmetic sequence: S = n/2*(a1 + an)"
    ],
    "methods": ["mathematical induction"],
    "symbolic_steps": [
        {"step": "Base Case", "reasoning": "For n = 1: 2 = 1(1 + 1) = 2  "},
        {"step": "Inductive Step", "reasoning": "Assume sum of first k even␣
 ↪numbers = k(k+1). Show for k+1."},
        {"step": "Expand S(k+1)", "reasoning": "S(k+1) = S(k) + 2(k+1) = k(k+1)␣
 ↪+ 2(k+1) = (k+1)(k+2)"}
    ],
    "conclusion": "The sum of the first n even numbers is n(n+1)"
}

#  Store it
print("\n Storing sketch...")
sketch_hash_id = store_proven_sketch(test_sketch)

#  Retrieve similar
print("\n Retrieving sketches related to 'even numbers'...")
hits = find_similar_by_problem("even numbers", memory)
print(f"  Found {len(hits)} match(es): {[h[:6] for h in hits]}")

#  Extract templates
print("\n Template distribution:")
templates = extract_templates(memory)
for key, entries in templates.items():
    print(f"   • {key} → {len(entries)}")

#  Generalize
print("\n Generalized sketch:")
generalized = generalize_sketch(test_sketch)
print(json.dumps(generalized, indent=2))
```

```
 Loading memory…
 Memory contains 0 sketches.

 Storing sketch…
 Stored sketch with hash:
0fe563698b392d64122977a45444fa5f1d60a63c17295091d816e2da517d4268

 Retrieving sketches related to 'even numbers'…
 Found 0 match(es): []

 Template distribution:

 Generalized sketch:
{
```

```
  "problem": "Prove that the sum of the first n even numbers is n(n+1)",
  "assumptions": [
    "Definition of even numbers: 2, 4, 6, …, 2n",
    "Formula for sum of arithmetic sequence: S = n/2*(a1 + an)"
  ],
  "methods": [
    "mathematical induction"
  ],
  "symbolic_steps": [
    {
      "step": "Base Case",
      "reasoning": "For n = 1: 2 = 1(1 + 1) = 2 \u2705"
    },
    {
      "step": "Inductive Step",
      "reasoning": "Assume sum of first k even numbers = k(k+1). Show for k+1."
    },
    {
      "step": "Expand S(k+1)",
      "reasoning": "S(k+1) = S(k) + 2(k+1) = k(k+1) + 2(k+1) = (k+1)(k+2)"
    }
  ],
  "conclusion": "The sum of the first n even numbers is n(n+1)"
}
```

0.7 Phase 6: Symbolic AST Compiler

[88]:
```python
# Phase 6: Symbolic AST Compiler - Final Implementation
from sympy import symbols, sympify, simplify
from sympy.core import Expr
from typing import Any, Dict, List, Tuple
import json
import re
import os

# --- Step 1: Parse a string into symbolic AST ---
def parse_expression_to_ast(expr_str: str) -> Expr:
    try:
        return sympify(expr_str)
    except Exception as e:
        print(f" Failed to parse expression: '{expr_str}'\nError: {e}")
        return None

# --- Step 2: Extract math expression from a sketch step ---
def extract_math_expr(step: Dict[str, str]) -> str:
    # Priority 1: explicit expression field
    if "expression" in step and step["expression"]:
```

```
        return step["expression"]

    # Priority 2: regex from reasoning text
    reasoning = step.get("reasoning", "")
    match = re.search(r"([a-zA-Z0-9_+\-*/^(). ]+)=([a-zA-Z0-9_+\-*/^(). ]+)",␣
 ↪reasoning)
    return match.group(0).strip() if match else ""

# --- Step 3: Compile a sketch to symbolic ASTs ---
def compile_sketch_to_ast(sketch: Dict[str, Any]) -> List[Tuple[str, Expr]]:
    ast_steps = []
    for step in sketch.get("symbolic_steps", []):
        expr_str = extract_math_expr(step)
        parsed = parse_expression_to_ast(expr_str)
        ast_steps.append((expr_str, parsed))
    return ast_steps

# --- Step 4: Visualize AST (optional) ---
def visualize_sympy_expr(expr: Expr) -> None:
    from sympy.printing.dot import dotprint
    from graphviz import Source
    dot = dotprint(expr)
    Source(dot).render(view=True)

# --- Step 5: Tree distance between expressions ---
def expr_tree_distance(expr1: Expr, expr2: Expr) -> int:
    if expr1 is None or expr2 is None:
        return float('inf')
    return sum(
        1 for a, b in zip(expr1.postorder_traversal(), expr2.
 ↪postorder_traversal())
        if a != b
    )

# --- Step 6: Mutate AST expression structurally ---
def mutate_expression(expr: Expr) -> Expr:
    from sympy import Add, Mul
    try:
        terms = list(expr.args)
        if isinstance(expr, Add):
            return Mul(*terms)
        elif isinstance(expr, Mul):
            return Add(*terms)
        return expr
    except Exception as e:
        print(" Mutation failed:", e)
        return expr
```

```python
# --- Step 7: Annotate sketch with symbolic ASTs ---
def annotate_sketch_with_ast(sketch: Dict[str, Any]) -> Dict[str, Any]:
    annotated = sketch.copy()
    for step in annotated.get("symbolic_steps", []):
        expr_str = extract_math_expr(step)
        parsed_expr = parse_expression_to_ast(expr_str)
        step["expression"] = expr_str
        step["ast_repr"] = str(parsed_expr) if parsed_expr is not None else None
    return annotated

# --- Step 8: Load, compile, and save updated sketch ---
def run_ast_phase_on_file(json_path: str, output_path: str = None):
    with open(json_path) as f:
        sketch = json.load(f)

    print(f"\n Loaded sketch: {json_path}")
    compiled = compile_sketch_to_ast(sketch)

    print("\n Parsed Symbolic ASTs:")
    for raw, parsed in compiled:
        print(f"• {raw} → {parsed}")

    updated = annotate_sketch_with_ast(sketch)
    output_path = output_path or json_path.replace(".json", "_phase6_ast.json")
    with open(output_path, "w") as f:
        json.dump(updated, f, indent=2)
    print(f"\n Phase 6 AST-enhanced sketch saved to {output_path}")

# --- Example usage ---
if __name__ == "__main__":
    run_ast_phase_on_file("hsde_phase1_output.json")
```

```
 Loaded sketch: hsde_phase1_output.json

 Parsed Symbolic ASTs:
• Eq(Sum(2*n - 1, (n, 1, 1)), 1**2) → Eq(Sum(2*n - 1, (n, 1, 1)), 1)
• Eq(Sum(2*n - 1, (n, 1, k)), k**2) → Eq(Sum(2*n - 1, (n, 1, k)), k**2)
• Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n - 1, (n, 1, k)) + (2*k + 1)) →
Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1, (n, 1, k)) + 1)
• Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k + 1) → Eq(Sum(2*n - 1, (n, 1, k +
1)), k**2 + 2*k + 1)
• Eq(k**2 + 2*k + 1, (k + 1)**2) → Eq(k**2 + 2*k + 1, (k + 1)**2)
• Eq(Sum(2*n - 1, (n, 1, k+1)), (k + 1)**2) → Eq(Sum(2*n - 1, (n, 1, k + 1)), (k
+ 1)**2)
```

```
Phase 6 AST-enhanced sketch saved to hsde_phase1_output_phase6_ast.json
```

0.8 Phase 7: Symbolic Similarity Graph

[94]:
```python
# Phase 7: Symbolic Similarity Graph and Rulial Navigation (Final Version)
import json
import networkx as nx
import sympy
from sympy.core import Expr
from typing import Dict, Any, List, Tuple
from pathlib import Path

# --- Load symbolic proof sketches from JSON files ---
def load_sketches_from_memory(directory: str) -> List[Dict[str, Any]]:
    return [json.load(open(p)) for p in Path(directory).glob("*.json")]

# --- Convert symbolic expression to token list (postorder traversal) ---
def tokenize_expr(expr: Expr) -> List[str]:
    tokens = []
    try:
        if isinstance(expr, sympy.Equality):
            tokens += [str(node) for node in expr.lhs.postorder_traversal()]
            tokens += [str(node) for node in expr.rhs.postorder_traversal()]
        else:
            tokens += [str(node) for node in expr.postorder_traversal()]
    except Exception as e:
        print(f"  Error during tokenization: {expr} → {e}")
    return tokens

# --- Compute structural difference between two expressions ---
def symbolic_distance(expr1: Expr, expr2: Expr) -> int:
    tokens1 = tokenize_expr(expr1)
    tokens2 = tokenize_expr(expr2)
    return sum(a != b for a, b in zip(tokens1, tokens2)) + abs(len(tokens1) -␣
 ↪len(tokens2))

# --- Build similarity graph from all sketch expressions ---
def build_symbolic_graph(sketches: List[Dict[str, Any]], distance_threshold=3)␣
 ↪-> nx.Graph:
    G = nx.Graph()
    node_index = 0
    ast_registry = []

    for sketch in sketches:
        for step in sketch.get("symbolic_steps", []):
            expr_str = step.get("expression")
            if not expr_str:
```

```
                continue
            try:
                expr = sympy.sympify(expr_str)

                if not isinstance(expr, sympy.Basic):
                    print(f"  Skipping non-symbolic: {expr_str} → {expr}")
                    continue

                G.add_node(node_index, expr=expr, label=expr_str)
                ast_registry.append((node_index, expr))
                node_index += 1

            except Exception as e:
                print(f"  Failed to parse: {expr_str} → {e}")
                continue

    # Add edges for similar expressions
    for i, (id1, expr1) in enumerate(ast_registry):
        for j, (id2, expr2) in enumerate(ast_registry[i+1:], i+1):
            dist = symbolic_distance(expr1, expr2)
            if dist <= distance_threshold:
                G.add_edge(id1, id2, weight=dist)

    return G

# --- Print graph summary ---
def print_graph_summary(G: nx.Graph):
    print(f"\n  Graph with {G.number_of_nodes()} nodes and {G.
 ↪number_of_edges()} edges\n")
    if G.number_of_edges() == 0:
        print("  No symbolic similarity links found (try increasing threshold).
 ↪")
        return

    print("  Top symbolic connections:")
    for u, v, data in sorted(G.edges(data=True), key=lambda x: x[2]['weight']):
        label_u = G.nodes[u]['label']
        label_v = G.nodes[v]['label']
        print(f"- {label_u}    {label_v}   [distance = {data['weight']}]")

# --- Main execution ---
if __name__ == "__main__":
    # You can change this to load many sketches
    sketches = [json.load(open("hsde_phase1_output_phase6_ast.json"))]
    G = build_symbolic_graph(sketches, distance_threshold=3)
    print_graph_summary(G)
```

```
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, 1)), 1) → 'Sum' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k)), k**2) → 'Sum' object
has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, 1)), 1) → 'Sum' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1,
(n, 1, k)) + 1) → 'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, 1)), 1) → 'Sum' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), k**2 + 2*k + 1) →
'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, 1)), 1) → 'Sum' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(k**2 + 2*k + 1, (k + 1)**2) → 'Add' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, 1)), 1) → 'Sum' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), (k + 1)**2) → 'Sum'
object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k)), k**2) → 'Sum' object
has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1,
(n, 1, k)) + 1) → 'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k)), k**2) → 'Sum' object
has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), k**2 + 2*k + 1) →
'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k)), k**2) → 'Sum' object
has no attribute 'postorder_traversal'
  Error during tokenization: Eq(k**2 + 2*k + 1, (k + 1)**2) → 'Add' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k)), k**2) → 'Sum' object
has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), (k + 1)**2) → 'Sum'
object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1,
(n, 1, k)) + 1) → 'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), k**2 + 2*k + 1) →
'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1,
(n, 1, k)) + 1) → 'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(k**2 + 2*k + 1, (k + 1)**2) → 'Add' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), 2*k + Sum(2*n - 1,
(n, 1, k)) + 1) → 'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), (k + 1)**2) → 'Sum'
object has no attribute 'postorder_traversal'
```

```
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), k**2 + 2*k + 1) →
'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(k**2 + 2*k + 1, (k + 1)**2) → 'Add' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), k**2 + 2*k + 1) →
'Sum' object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), (k + 1)**2) → 'Sum'
object has no attribute 'postorder_traversal'
  Error during tokenization: Eq(k**2 + 2*k + 1, (k + 1)**2) → 'Add' object has
no attribute 'postorder_traversal'
  Error during tokenization: Eq(Sum(2*n - 1, (n, 1, k + 1)), (k + 1)**2) → 'Sum'
object has no attribute 'postorder_traversal'

  Graph with 6 nodes and 15 edges

  Top symbolic connections:
- Eq(Sum(2*n - 1, (n, 1, 1)), 1**2)     Eq(Sum(2*n - 1, (n, 1, k)), k**2)
[distance = 0]
- Eq(Sum(2*n - 1, (n, 1, 1)), 1**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n -
1, (n, 1, k)) + (2*k + 1))   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, 1)), 1**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k
+ 1)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, 1)), 1**2)     Eq(k**2 + 2*k + 1, (k + 1)**2)
[distance = 0]
- Eq(Sum(2*n - 1, (n, 1, 1)), 1**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), (k +
1)**2)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k)), k**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n -
1, (n, 1, k)) + (2*k + 1))   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k)), k**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k
+ 1)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k)), k**2)     Eq(k**2 + 2*k + 1, (k + 1)**2)
[distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k)), k**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), (k +
1)**2)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n - 1, (n, 1, k)) + (2*k + 1))
Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k + 1)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n - 1, (n, 1, k)) + (2*k + 1))    Eq(k**2
+ 2*k + 1, (k + 1)**2)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k+1)), Sum(2*n - 1, (n, 1, k)) + (2*k + 1))
Eq(Sum(2*n - 1, (n, 1, k+1)), (k + 1)**2)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k + 1)     Eq(k**2 + 2*k + 1, (k +
1)**2)   [distance = 0]
- Eq(Sum(2*n - 1, (n, 1, k+1)), k**2 + 2*k + 1)     Eq(Sum(2*n - 1, (n, 1,
k+1)), (k + 1)**2)   [distance = 0]
- Eq(k**2 + 2*k + 1, (k + 1)**2)     Eq(Sum(2*n - 1, (n, 1, k+1)), (k + 1)**2)
[distance = 0]
```

0.9 Phase 8: Rewrite Engine

[95]:
```python
# Phase 8: Symbolic Rewrite Engine (HSDE)
import sympy
from sympy import symbols, simplify, Wild, Eq
from sympy.core import Expr
from typing import List, Tuple, Callable

# --- Define rewrite rules as (pattern, replacement) ---
def get_rewrite_rules() -> List[Tuple[Expr, Expr]]:
    a, b, x, k = symbols('a b x k')
    A = Wild('A')
    B = Wild('B')
    K = Wild('K')

    return [
        (A**2 + 2*A*B + B**2, (A + B)**2),
        (A**2 - 2*A*B + B**2, (A - B)**2),
        (K**2 + 2*K + 1, (K + 1)**2),
        (K*(K + 1) + 2*(K + 1), (K + 1)*(K + 2)),
        ((K + 1)*(K + 2), K*(K + 1) + 2*(K + 1)),  # reverse
    ]

# --- Apply all rewrite rules to an expression ---
def apply_rewrites(expr: Expr, rules: List[Tuple[Expr, Expr]]) -> List[Expr]:
    variants = []
    for pattern, replacement in rules:
        try:
            new_expr = expr.replace(pattern, replacement)
            if new_expr != expr:
                variants.append(new_expr)
        except Exception as e:
            print(f"  Rewrite error: {e}")
    return variants

# --- Score how similar a rewrite is to the original ---
def score_rewrite(expr: Expr, target: Expr) -> float:
    try:
        return 1.0 if expr.equals(target) else 0.0
    except:
        return 0.0

# --- Example run ---
if __name__ == "__main__":
    k = symbols("k")
    original = Eq(k**2 + 2*k + 1, (k + 1)**2)
    rules = get_rewrite_rules()
```

```
    lhs = original.lhs
    variants = apply_rewrites(lhs, rules)

    print("  Original:", lhs)
    print("  Rewrites:")
    for v in variants:
        print("→", v, "| equals RHS:", v.equals(original.rhs))
```

```
  Original: k**2 + 2*k + 1
  Rewrites:
→ (k + 1)**2 | equals RHS: True
→ (-k - 1)**2 | equals RHS: True
→ (k + 1)**2 | equals RHS: True
```

[ ]: